

Laporan Tugas Besar 2

TBFO (IF2224)



Disusun Oleh

Kelompok GooBurs:

Eliana Natalie Widjojo (13524116)

Muhammad Rafi Akbar (13524125)

Varistha Devi (13524135)

Ahmad Rinofaros Muchtar (13524138)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

BANDUNG 2025

Daftar Isi

Daftar Isi	2
Bab 1. Teori Singkat	3
1. Parser	3
2. Parse Tree	3
3. Recursive Descent Parsing	4
4. Context-Free Grammar	5
5. Top-Down Parsing	5
6. Lookahead	6
Bab 2. Perancangan dan Implementasi	8
Bab 3. Pengujian	9
Bab 4. Kesimpulan dan Saran	28
Lampiran	29
Referensi	29

Bab 1. Teori Singkat

1. Parser

Parser atau Syntax Analyzer adalah sebuah program atau komponen dalam proses kompilasi yang menerima deretan token hasil Lexical Analyzer (lexer) dan menganalisis struktur gramatikalnya untuk menghasilkan sebuah representasi hierarkis dalam sebuah struktur pohon, yakni parse tree. Ia bekerja dengan memverifikasi urutan token yang diterima sesuai dengan aturan tata bahasa (grammar) dari bahasa pemrograman yang bersangkutan, sekaligus membangun struktur pohon yang merepresentasikan hubungan logis antar elemen program tersebut.

Dalam pipeline kompilasi, parser memiliki hubungan producer-consumer dengan lexer. Lexer berperan sebagai producer yang membaca source code mentah karakter per karakter dan mengubahnya menjadi aliran token, seperti keyword, identifier, dan operator. Parser kemudian berperan sebagai consumer yang mengambil aliran token tersebut secara berurutan untuk dianalisis struktur gramatikalnya dan diubah menjadi parse tree, sekaligus memastikan apakah susunan token-token tersebut membentuk program yang valid.

Dalam menjalankan tugasnya, parser memiliki lima peran utama. Pertama, melakukan analisis sintaks berbasis context-free grammar dengan memverifikasi apakah struktur kode mengikuti aturan gramatikal yang telah didefinisikan, seperti memastikan setiap begin diakhiri end. Kedua, memandu analisis context-sensitive dengan menyediakan struktur yang diperlukan oleh tahap semantic analysis untuk verifikasi aturan berbasis makna seperti pengecekan tipe data. Ketiga, membangun representasi antara (intermediate representation) berupa parse tree atau AST yang menjadi masukan bagi tahap kompilasi selanjutnya. Keempat, menghasilkan pesan error yang informatif dengan mengidentifikasi dan melaporkan syntax error secara jelas, termasuk menyebutkan baris dan token yang bermasalah. Kelima, mencoba pemulihan dari error (error recovery) dengan berusaha melanjutkan proses parsing meskipun ditemukan error, sehingga lebih dari satu error dapat terdeteksi dalam satu kali proses.

2. Parse Tree

Parse tree, atau yang juga disebut sebagai derivation tree, adalah representasi grafis dari proses derivasi suatu string berdasarkan aturan produksi dalam sebuah CFG (Context-Free Grammar). Struktur ini menunjukkan bagaimana simbol awal (start

symbol) dapat ditransformasikan menjadi rangkaian simbol terminal melalui penerapan aturan produksi secara berurutan.

Dalam parse tree, setiap node merepresentasikan simbol dalam grammar, baik berupa terminal maupun non-terminal. Sementara itu, hubungan antar node yang digambarkan melalui edge (garis penghubung) menunjukkan penerapan aturan produksi yang mengubah suatu simbol menjadi simbol-simbol lainnya. Dengan demikian, parse tree membantu memvisualisasikan struktur sintaks dari suatu string secara hierarkis.

3. Recursive Descent Parsing

Recursive descent parsing adalah metode syntax analysis top-down yang menggunakan sekumpulan prosedur atau fungsi rekursif untuk memproses input, di mana setiap non-terminal dalam grammar direpresentasikan oleh satu fungsi tertentu. Parser bekerja dengan membaca input dari kiri ke kanan dan mencoba membangun parse tree berdasarkan aturan produksi yang ada, mengikuti leftmost derivation. Dengan demikian, proses parsing dapat dipandang sebagai penyusunan pohon secara bertahap dari akar hingga ke daun (preorder), sesuai dengan struktur grammar. Metode ini relatif sederhana untuk diimplementasikan, terutama pada bahasa dengan struktur yang tidak terlalu kompleks.

Dalam praktiknya, recursive descent parser dapat menggunakan teknik backtracking, yaitu mencoba beberapa aturan produksi dan kembali ke langkah sebelumnya jika terjadi kegagalan dalam pencocokan. Namun, pendekatan ini bisa menjadi tidak efisien. Oleh karena itu, terdapat bentuk khusus yang lebih optimal, yaitu predictive parser, yang merupakan recursive descent parser tanpa backtracking. Predictive parser membuat keputusan parsing hanya berdasarkan satu simbol lookahead (biasanya pada grammar LL(1)), sehingga prosesnya lebih cepat dan deterministik.

Agar dapat digunakan secara efektif, grammar yang dipakai dalam recursive descent parser harus memenuhi beberapa syarat, seperti tidak mengandung left recursion dan telah melalui proses left factoring. Jika grammar masih memiliki left recursion, maka perlu dilakukan transformasi terlebih dahulu. Misalnya, aturan seperti $E \rightarrow E + T \mid T$ dapat diubah menjadi bentuk yang tidak rekursif ke kiri, seperti $E \rightarrow T E'$ dan $E' \rightarrow + T E' \mid \epsilon$. Transformasi ini memungkinkan parser untuk bekerja tanpa mengalami infinite recursion.

Secara umum, algoritma recursive descent parser bekerja dengan memilih suatu produksi untuk simbol awal, kemudian memproses setiap simbol di sisi kanan produksi tersebut secara berurutan. Jika simbol tersebut adalah non-terminal, maka prosedur yang sesuai akan dipanggil secara rekursif. Jika simbol adalah terminal, maka parser akan mencocokkannya dengan input saat ini. Jika terjadi ketidaksesuaian,

parser dapat melakukan backtracking (jika diizinkan) atau langsung menghasilkan error. Dengan mekanisme ini, recursive descent parsing menjadi salah satu teknik parsing yang intuitif dan banyak digunakan dalam implementasi compiler sederhana.

4. Context-Free Grammar

Context-Free Grammar (CFG) adalah suatu sistem formal yang digunakan untuk mendeskripsikan himpunan bahasa yang disebut context-free languages (CFL). CFG terdiri dari sekumpulan aturan produksi yang mendefinisikan semua kemungkinan string dalam suatu bahasa formal. Aturan-aturan ini menjelaskan bagaimana simbol-simbol dalam bahasa dapat dikombinasikan dan ditransformasikan untuk menghasilkan string yang valid. Dengan demikian, CFG berfungsi untuk menghasilkan seluruh string dalam suatu bahasa menggunakan aturan yang terstruktur, serta memiliki kemampuan yang lebih kuat dibandingkan regular expression dan finite automata.

Secara formal, sebuah CFG dilambangkan sebagai G yang terdiri dari empat komponen utama. Pertama, V yaitu himpunan hingga dari variabel atau simbol non-terminal yang berfungsi sebagai representasi kategori abstrak. Kedua, T yaitu himpunan hingga dari simbol terminal yang merupakan simbol nyata atau alfabet dalam bahasa. Ketiga, S yaitu simbol awal (start symbol), yaitu salah satu simbol non-terminal dari V yang menjadi titik awal dalam proses derivasi. Keempat, P yaitu himpunan hingga dari aturan produksi, yang umumnya berbentuk $A \rightarrow \alpha$, di mana A adalah simbol non-terminal dan α adalah rangkaian simbol terminal dan/atau non-terminal.

Dalam CFG, simbol non-terminal digunakan sebagai placeholder atau representasi struktur (misalnya E atau S), sedangkan simbol terminal merupakan karakter aktual seperti huruf, angka, atau operator (misalnya a , b , $+$, $*$, $($, $)$). Aturan produksi menentukan bagaimana suatu non-terminal dapat digantikan dengan kombinasi simbol lain, sehingga membentuk struktur bahasa yang diinginkan. Seluruh proses pembentukan string selalu dimulai dari simbol awal dan dilanjutkan dengan penerapan aturan produksi hingga hanya tersisa simbol terminal.

5. Top-Down Parsing

Top-down parsing adalah metode analisis sintaks yang membangun parse tree mulai dari simbol awal (root) hingga ke daun (simbol terminal) dengan cara mencocokkan string input secara bertahap. Proses ini dimulai dari start symbol, kemudian parser memperluas non-terminal menggunakan aturan produksi hingga seluruh input berhasil dicocokkan. Pendekatan ini mengikuti leftmost derivation, di mana non-terminal paling kiri selalu dikembangkan terlebih dahulu.

Metode ini juga dikenal dengan nama recursive descent parsing atau predictive parsing, dan umumnya lebih mudah diimplementasikan serta cocok digunakan untuk bahasa yang sederhana. Namun, top-down parsing dapat mengalami kesulitan ketika menghadapi grammar yang kompleks atau ambigu karena proses pencocokan bisa menjadi tidak efisien atau tidak deterministik.

Dalam implementasinya, top-down parsing dibagi menjadi dua jenis, yaitu dengan backtracking dan tanpa backtracking. Pada metode dengan backtracking, parser dapat mencoba beberapa aturan produksi dan kembali ke langkah sebelumnya jika terjadi kegagalan, sehingga mampu menangani lebih banyak kemungkinan, tetapi cenderung lambat. Sementara itu, metode tanpa backtracking tidak mengulang pilihan yang sudah diambil sehingga lebih cepat dan efisien, tetapi hanya dapat digunakan pada grammar tertentu seperti LL grammar.

Agar top-down parsing dapat bekerja dengan baik, grammar yang digunakan harus bebas dari left recursion, sudah melalui proses left factoring, dan tidak ambigu sehingga keputusan parsing dapat dilakukan secara deterministik. Perlu juga diketahui bahwa top-down parsing menggunakan leftmost derivation, sedangkan bottom-up parsing menggunakan reverse rightmost derivation dalam prosesnya.

6. Lookahead

LL(k) parser merupakan salah satu jenis top-down parser yang bekerja dengan membaca input dari kiri ke kanan (left to right) dan membangun leftmost derivation. Huruf “k” pada LL(k) menunjukkan jumlah simbol lookahead yang dapat digunakan oleh parser untuk membantu menentukan aturan produksi yang tepat. Dalam pendekatan ini, parser tidak hanya bergantung pada simbol saat ini, tetapi juga dapat mempertimbangkan beberapa simbol berikutnya sebelum mengambil keputusan parsing.

Konsep utama yang relevan dalam implementasi ini adalah lookahead, yaitu kemampuan parser untuk “mengintip” sejumlah token ke depan tanpa langsung mengonsumsinya. Penting untuk dipahami bahwa lookahead tidak berarti parser memproses banyak token sekaligus; parser tetap mengonsumsi input satu per satu. Namun, dengan adanya lookahead, parser dapat membuat keputusan yang lebih akurat terkait cabang produksi yang harus dipilih, sehingga mengurangi kemungkinan kesalahan atau kebutuhan backtracking.

Dalam praktiknya, penggunaan fungsi seperti peekAt() mencerminkan penerapan konsep lookahead tersebut. Fungsi ini memungkinkan parser untuk mengakses token ke-i berikutnya tanpa memindahkan posisi pembacaan input. Dengan demikian, meskipun parser yang diimplementasikan tidak sepenuhnya mengikuti formalitas LL(k) parser (misalnya tanpa tabel parsing atau analisis FIRST/FOLLOW secara eksplisit), mekanisme lookahead yang digunakan tetap sejalan dengan prinsip dasar

LL(k), yaitu memanfaatkan informasi dari beberapa simbol ke depan untuk meningkatkan ketepatan proses parsing.

Bab 2. Perancangan dan Implementasi

1. lex_analyzer

Lexical Analyzer memindai file sumber karakter per karakter menggunakan mekanisme DFA, mengubahnya menjadi aliran token, dan melacak nomor baris tiap token. Program ini memiliki 4 komponen utama, yakni:

1. Manajemen Karakter

Lexer membaca input melalui `std::ifstream`. Terdapat tiga fungsi inti: `readChar()` yang membaca satu karakter dari file dan menginkremen `currentLine` setiap kali menemukan newline `\n`; `unreadChar(char c)` yang mengembalikan satu karakter ke stream menggunakan `putback()` untuk memungkinkan backtrack saat lookahead diperlukan; serta `skipWhitespace()` yang mengonsumsi spasi, tab, dan newline tanpa menghasilkan token sekaligus melacak perpindahan baris.

2. Loop Tokenisasi

Fungsi `tokenize()` adalah entry point utama yang memanggil `nextToken()` berulang kali hingga EOF, memfilter token bertipe COMMENT, dan menyisipkan token bertipe BLANK_LINE sebagai penanda baris kosong. Hasil akhirnya berupa `std::vector<Token>` yang diteruskan ke parser.

3. Pengenalan Token

Fungsi `nextToken()` membaca karakter pertama dari input lalu mendelegasikan ke fungsi spesialis. Setiap fungsi spesialis mengimplementasikan state machine DFA menggunakan enum State. Prinsip longest-match diterapkan: lexer terus membaca selama karakter berikutnya masih valid, kemudian berhenti dan memanggil `unreadChar()` saat state tidak dapat berlanjut. Pengenalan keyword dilakukan oleh `checkKeyword()` menggunakan `std::unordered_map` dengan pencocokan case-insensitive melalui fungsi `toLowerCase()`.

4. Penanganan Komentar

Dua format komentar didukung: kurung kurawal `{ ... }` yang ditangani oleh `readBraceComment()`, dan format tanda bintang dalam kurung `(* ... *)` yang ditangani oleh `readParenComment()`. Keduanya menghasilkan token COMMENT yang kemudian difilter dari output.

Selain itu, Lexer hanya menghasilkan token-token berikut:

1. **Literal:** INTCON, REALCON, CHARCON, STRING
2. **Identifier:** IDENT

3. **Keyword:** PROGRAMSY, VARSY, CONSTSY, TYPESY, FUNCTIONSY, PROCEDURESY, ARRAYSY, RECORDSY, BEGINSY, ENDSY, IFSY, THENSY, ELSESY, CASESY, OFSY, WHILESY, DOSY, REPEATSY, UNTILSY, FORSY, TOSY, DOWNTOSY, NOTSY, ANDSY, ORSY
4. **Operator:** PLUS, MINUS, TIMES, RDIV, IDIV, IMOD, EQL, NEQ, GTR, GEQ, LSS, LEQ, BECOMES
5. **Delimiter:** LPARENT, RPARENT, LBRACK, RBRACK, COMMA, SEMICOLON, PERIOD, COL
6. **Special:** COMMENT (difilter), BLANK_LINE (metadata), UNKNOWN, EOF_TOKEN

2. parser

Parser menerima aliran token dari `lex_analyzer`, memeriksa dan membandingkan urutan token dengan grammar bahasa Arion, dan menghasilkan sebuah parse tree. Berikut fungsi-fungsi yang digunakan dalam file beserta tujuannya:

1. **Constructor Parser(const std::vector<Token>& rawTokens)** menerima seluruh token dari lexer dan memfilter token bertipe COMMENT dan BLANK_LINE. Token yang tersisa disimpan dalam `std::vector<Token>` tokens dengan pointer posisi pos yang diinisialisasi ke 0.
2. **Util Functions**
 - a. **Manajemen Posisi Token:** `peek()` mengembalikan token pada posisi saat ini tanpa mengonsumsinya; `peekAt(int offset)` mengembalikan token pada posisi `pos + offset` untuk implementasi lookahead LL(k); `advance()` mengonsumsi dan mengembalikan token saat ini.
 - b. **Konsumsi Token dengan Validasi:** `expect(TokenType type)` melempar `ParseError` jika token tidak sesuai dengan pesan "Syntax error at line N: unexpected token 'X', expected 'Y'"; `check(TokenType type)` memeriksa tipe token saat ini tanpa mengonsumsi; `checkAny()` memeriksa terhadap beberapa tipe; `match()` menggabungkan `check` dan `advance`; `error()` melempar `ParseError` secara eksplisit; `isAtEnd()` memeriksa apakah posisi sudah mencapai EOF_TOKEN.
 - c. **Token Label:** `tokenLabel()` mengonversi token ke representasi string untuk label node terminal di parse tree.
 - d. **Pembuatan Node Terminal:** `makeTerminal()` membuat `ParseNode` baru dengan label dari `tokenLabel()`, dipanggil setiap kali token terminal dikonsumsi dan perlu dimasukkan ke dalam pohon.
 - e. **Entry Point Parsing:** `parse()` memanggil `parseProgram()` sebagai titik masuk dan mengembalikan root `ParseNode*`; `printTree()` mencetak parse tree secara rekursif ke output stream menggunakan karakter box-drawing Unicode.

3. **Grammar** Seluruh grammar di-hardcode sebagai fungsi-fungsi rekursif. Setiap non-terminal direpresentasikan oleh satu fungsi sesuai prinsip recursive descent parsing.
- a. **Program Structure dan Deklarasi:** `parseProgram()`, `parseProgramHeader()`, `parseDeclarationPart()`, `parseConstDeclaration()`, `parseConstant()`, `parseTypeDeclaration()`, `parseType()`, `parseArrayType()`, `parseRange()`, `parseEnumerated()`, `parseRecordType()`, `parseFieldList()`, `parseFieldPart()`, `parseVarDeclaration()`, `parseIdentifierList()`, `parseSubprogramDeclaration()`, `parseProcedureDeclaration()`, `parseFunctionDeclaration()`, `parseFormalParameterList()`, `parseParameterGroup()`, `parseBlock()`.
 - b. **Statements:** `parseCompoundStatement()`, `parseStatementList()`, `parseStatement()`, `parseAssignmentStatement()`, `parseIfStatement()`, `parseCaseStatement()`, `parseCaseBlock()`, `parseWhileStatement()`, `parseRepeatStatement()`, `parseForStatement()`. Mekanisme disambiguasi menggunakan `peekAt(1)` untuk membedakan assignment (diikuti BECOMES) dan procedure/function call (diikuti LPARENT atau token lain), merupakan implementasi lookahead LL(2).
 - c. **Expressions:** `parseExpression()`, `parseSimpleExpression()`, `parseTerm()`, `parseFactor()`, `parseRelationalOperator()`, `parseAdditiveOperator()`, `parseMultiplicativeOperator()`.
 - d. **Variable & Component Access:** `parseVariable()`, `parseComponentVariable()`, `parseIndexList()`, `parseProcedureFunctionCall()`, `parseParameterList()`.

Bab 3. Pengujian

1. Kasus 1

Input:

```
program Hello;  
  
var  
  a, b: integer;  
  
begin  
  a := 5;  
  b := a + 10;  
  writeln('Result = ', b);  
end.
```

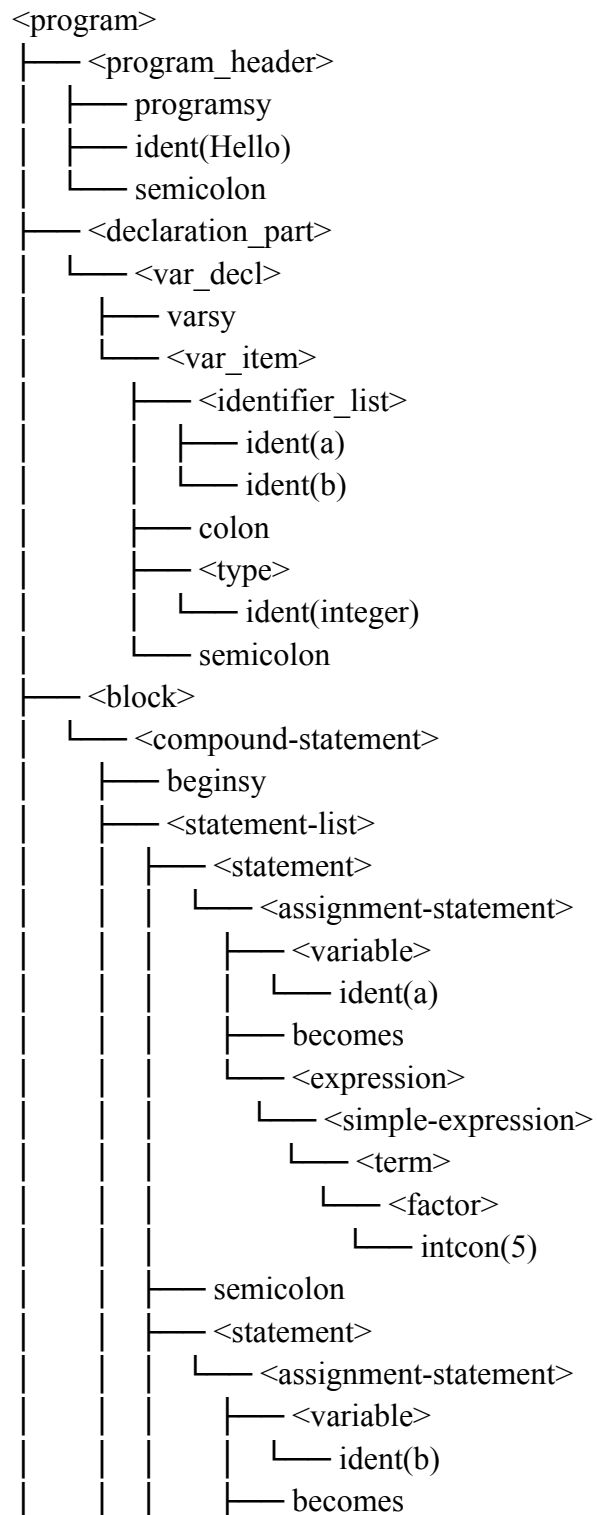
Output (di terminal, untuk .txt outputnya hanya menyimpan parse tree):

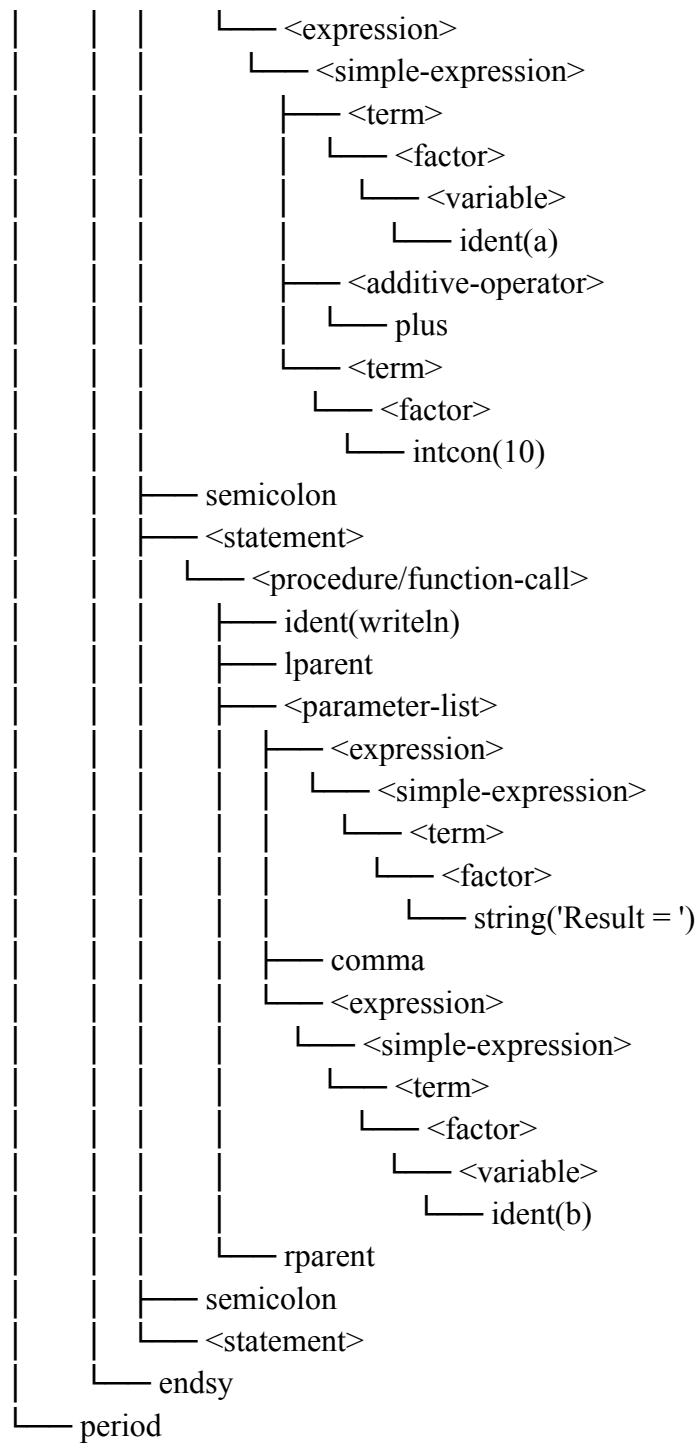
```
programsy  
ident (Hello)  
semicolon  
  
varsy  
ident (a)  
comma  
ident (b)  
colon  
ident (integer)  
semicolon  
  
beginsy  
ident (a)  
becomes  
intcon (5)  
semicolon  
ident (b)  
becomes  
ident (a)  
plus  
intcon (10)  
semicolon  
ident (writeln)  
lparent  
string ('Result = ')  
comma
```

ident (b)
rparent
semicolon

endsy
period

--- PARSE TREE ---





2. Kasus 2

Input:

program SpacingTest;

var

x, y: integer;

begin

```
x := 1;

y := x + 2;
writeln('x = ', x);
writeln('y = ', y);

end.
```

Output (di terminal, untuk .txt outputnya hanya menyimpan parse tree):

```
programsy
ident (SpacingTest)
semicolon
```

```
varsy
ident (x)
comma
ident (y)
colon
ident (integer)
semicolon
```

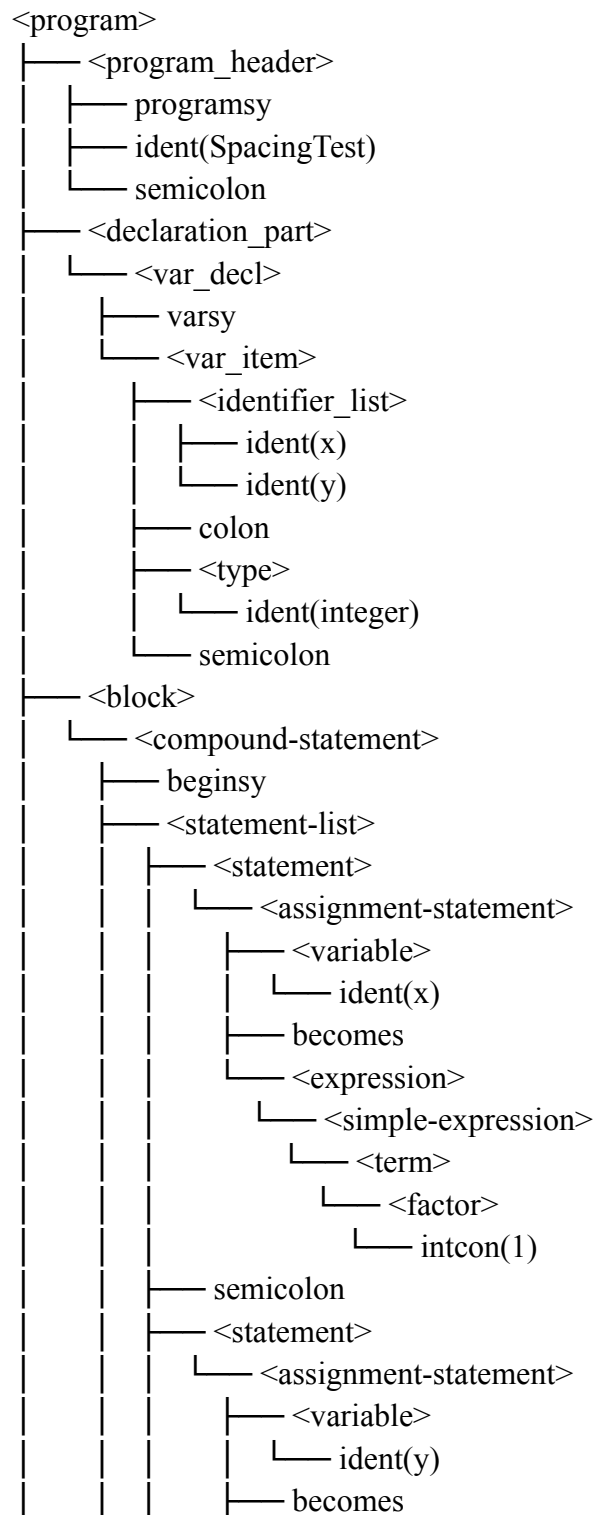
```
beginsy
ident (x)
becomes
intcon (1)
semicolon
```

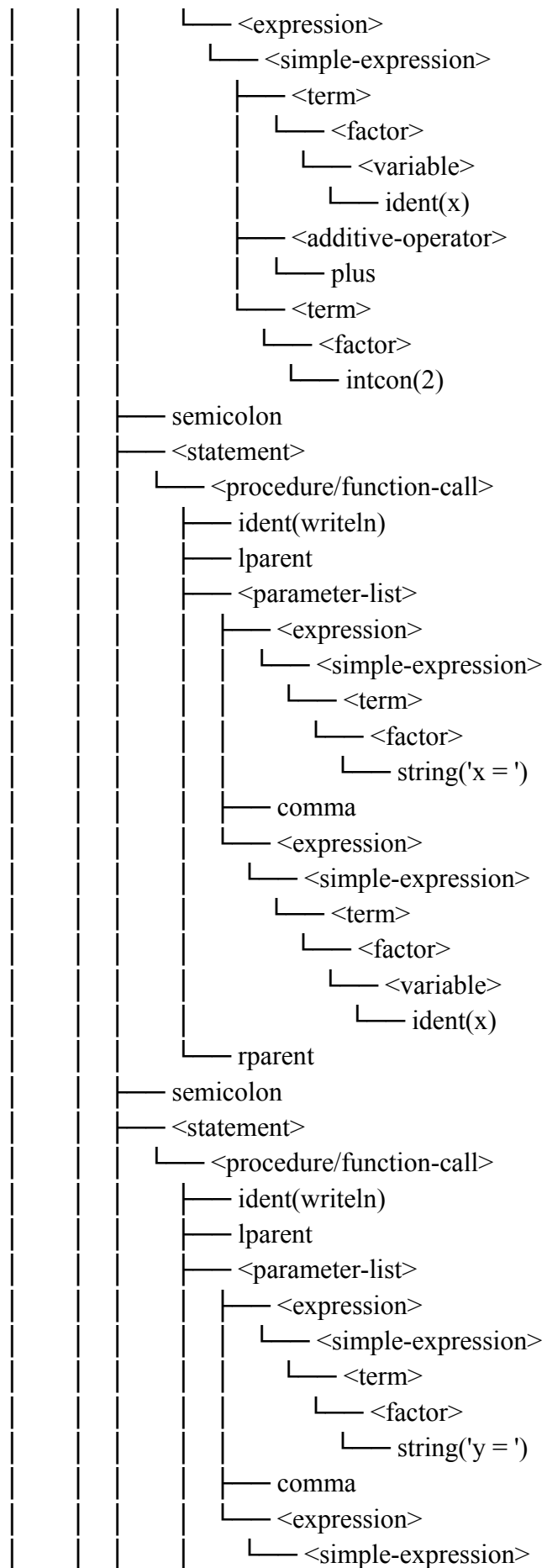
```
ident (y)
becomes
ident (x)
plus
intcon (2)
semicolon
ident (writeln)
lparent
string ('x = ')
comma
ident (x)
rparent
semicolon
ident (writeln)
lparent
string ('y = ')
comma
```

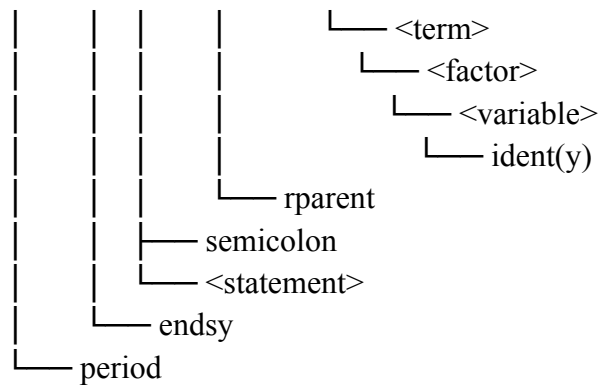
ident (y)
rparent
semicolon

endsy
period

--- PARSE TREE ---







3. Kasus 3

Input:

Program MixCase;

var

Alpha1, beta2: integer;

begin

{ komentar gaya brace }

Alpha1 := 12;

(* komentar gaya parenthesis-star *)

beta2 := Alpha1 + 3;

end.

Output (di terminal, untuk .txt outputnya hanya menyimpan parse tree):

programsy

ident (MixCase)

semicolon

varsy

ident (Alpha1)

comma

ident (beta2)

colon

ident (integer)

semicolon

beginsy

ident (Alpha1)

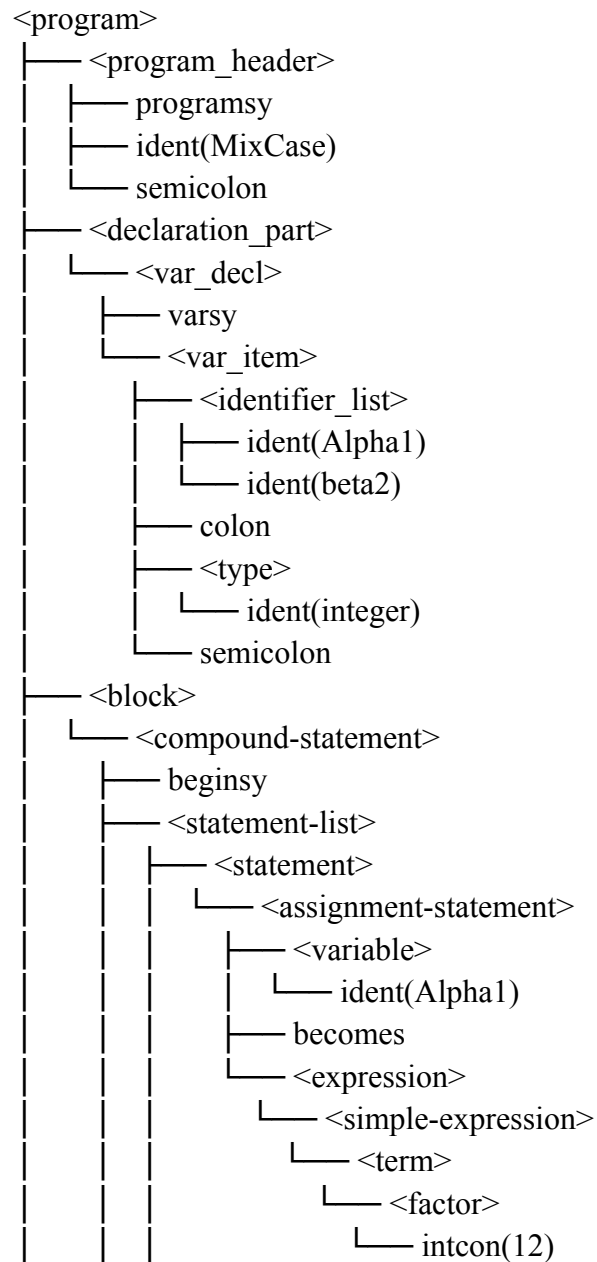
becomes

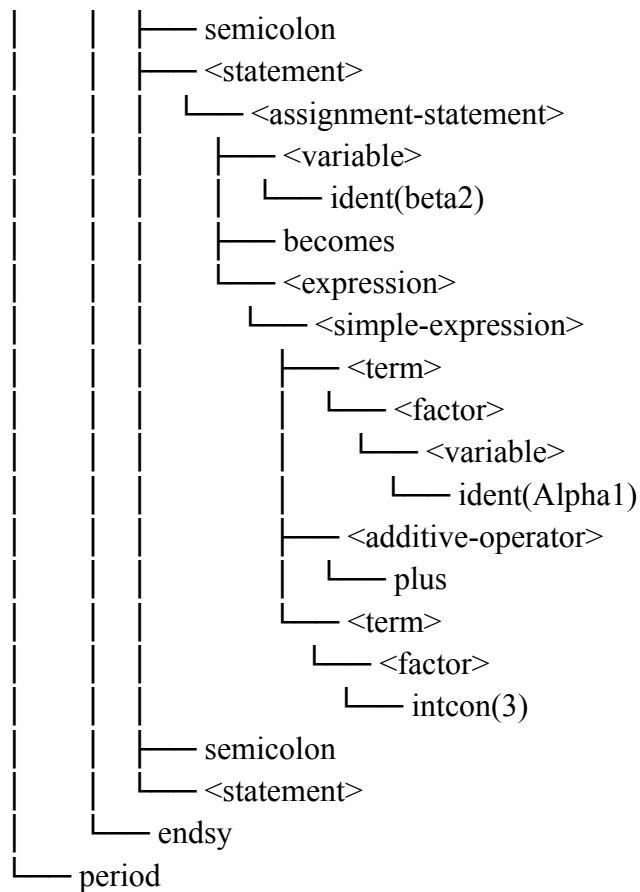
intcon (12)
semicolon

ident (beta2)
becomes
ident (Alpha1)
plus
intcon (3)
semicolon

endsy
period

--- PARSE TREE ---





4. Kasus 4

Input:

```
program LiteralsTest;
```

```
var
```

```
  x: integer;
```

```
begin
```

```
  x := 7;
```

```
  writeln('A');
```

```
  writeln('I'm here');
```

```
  writeln('value = ', 3.1415);
```

```
end.
```

Output (di terminal, untuk .txt outputnya hanya menyimpan parse tree):

```
programsy
```

```
ident (LiteralsTest)
```

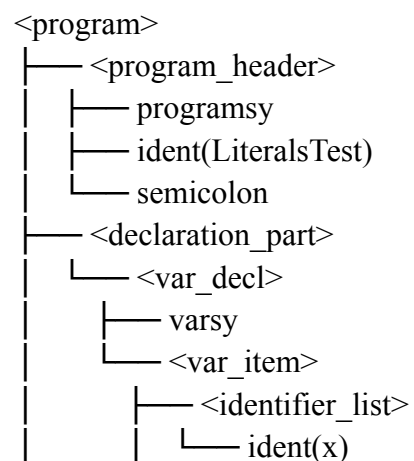
```
semicolon
```

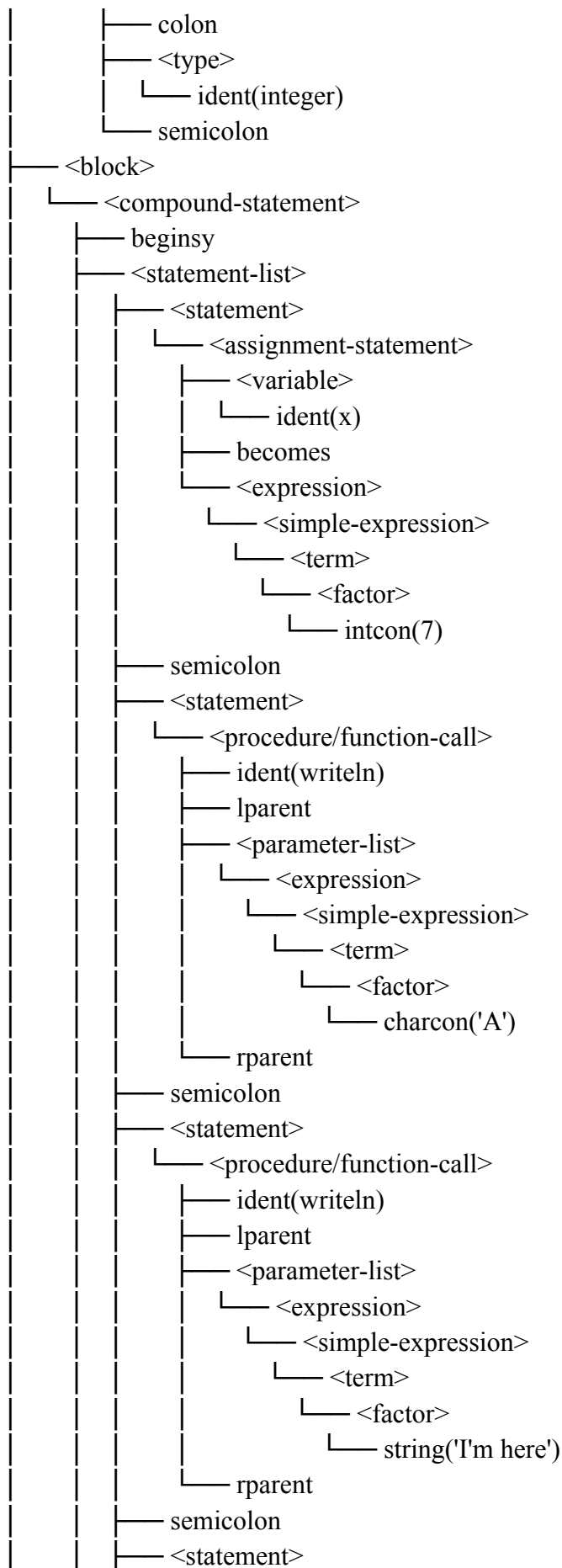
varsy
ident (x)
colon
ident (integer)
semicolon

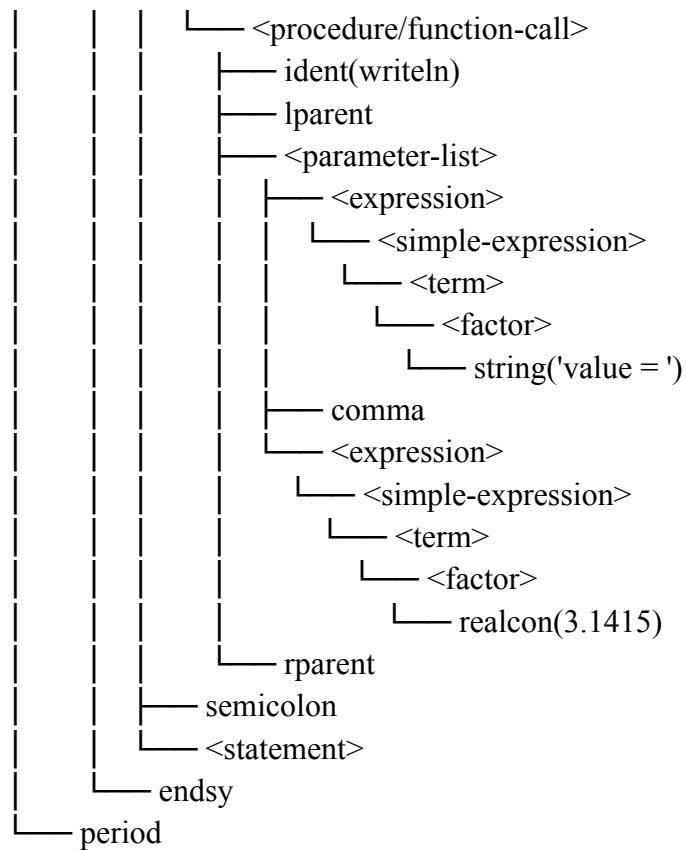
beginsy
ident (x)
becomes
intcon (7)
semicolon
ident (writeln)
lparent
charcon ('A')
rparent
semicolon
ident (writeln)
lparent
string ('I'm here')
rparent
semicolon
ident (writeln)
lparent
string ('value = ')
comma
realcon (3.1415)
rparent
semicolon

endsy
period

--- PARSE TREE ---







5. Kasus 5

Input:

```

program CompareOps;

var
  a, b: integer;

begin
  a := 10;
  b := 20;
  if a <= b then writeln('leq');
  if a <> b then writeln('neq');
  if b >= a then writeln('geq');
  if a < b then writeln('lss');
  if b > a then writeln('gtr');
  if a == a then writeln('eql');
end.

```

Output (di terminal, untuk .txt outputnya hanya menyimpan parse tree):

programsy
ident (CompareOps)
semicolon

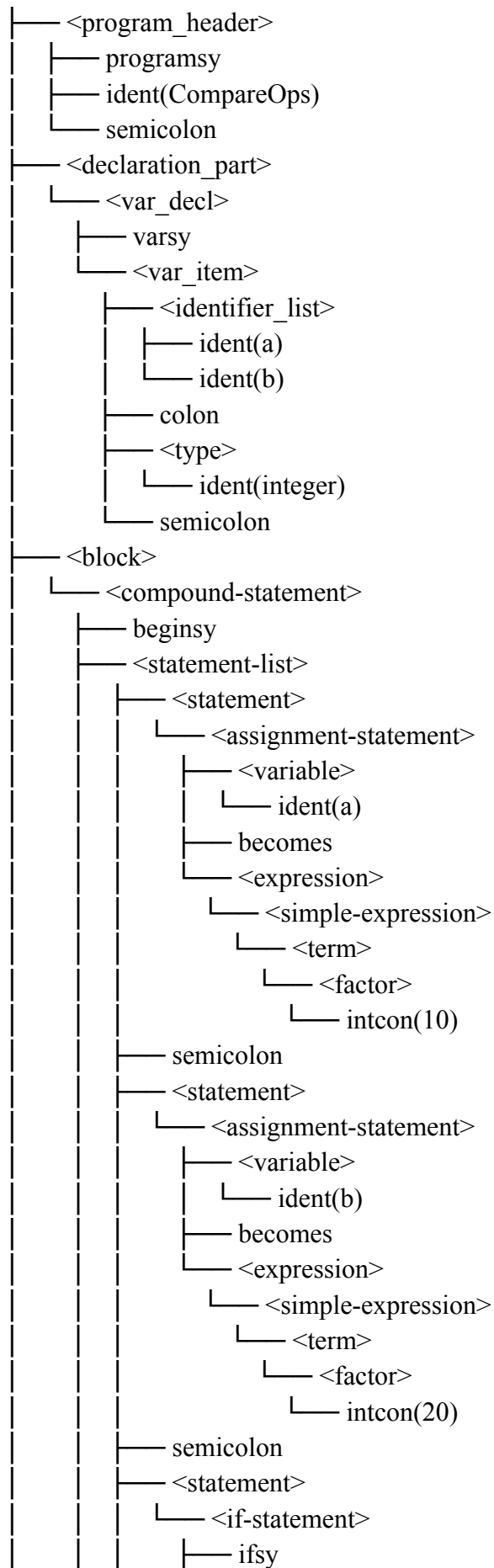
varsy
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon

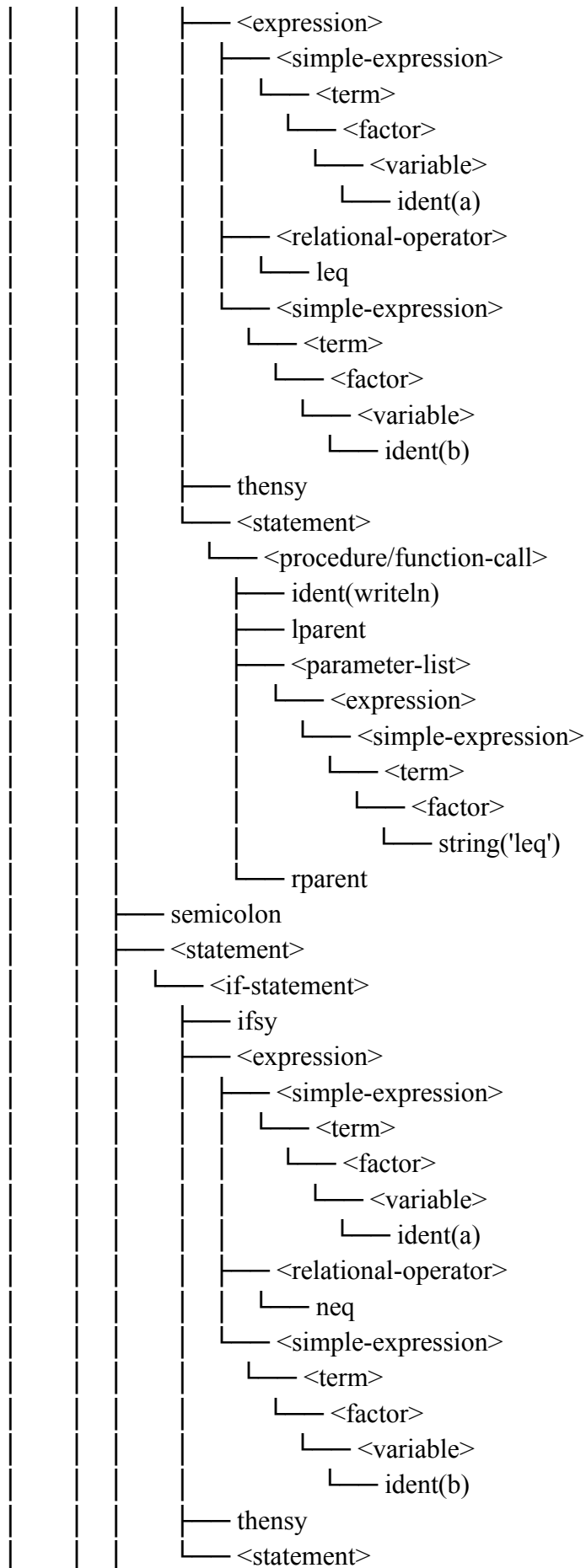
beginsy
ident (a)
becomes
intcon (10)
semicolon
ident (b)
becomes
intcon (20)
semicolon
ifsy
ident (a)
leq
ident (b)
thensy
ident (writeln)
lparent
string ('leq')
rparent
semicolon
ifsy
ident (a)
neq
ident (b)
thensy
ident (writeln)
lparent
string ('neq')
rparent
semicolon
ifsy
ident (b)

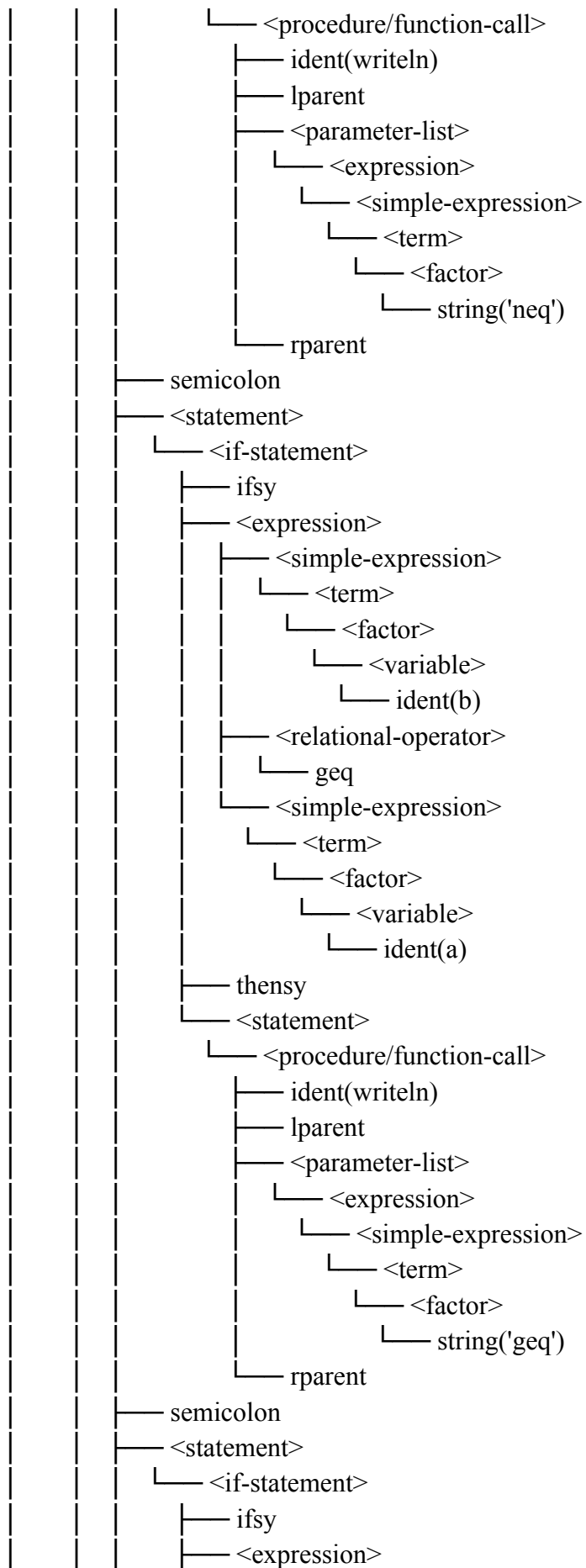
geq
ident (a)
thensy
ident (writeln)
lparent
string ('geq')
rparent
semicolon
ifsy
ident (a)
lss
ident (b)
thensy
ident (writeln)
lparent
string ('lss')
rparent
semicolon
ifsy
ident (b)
gtr
ident (a)
thensy
ident (writeln)
lparent
string ('gtr')
rparent
semicolon
ifsy
ident (a)
eql
ident (a)
thensy
ident (writeln)
lparent
string ('eql')
rparent
semicolon

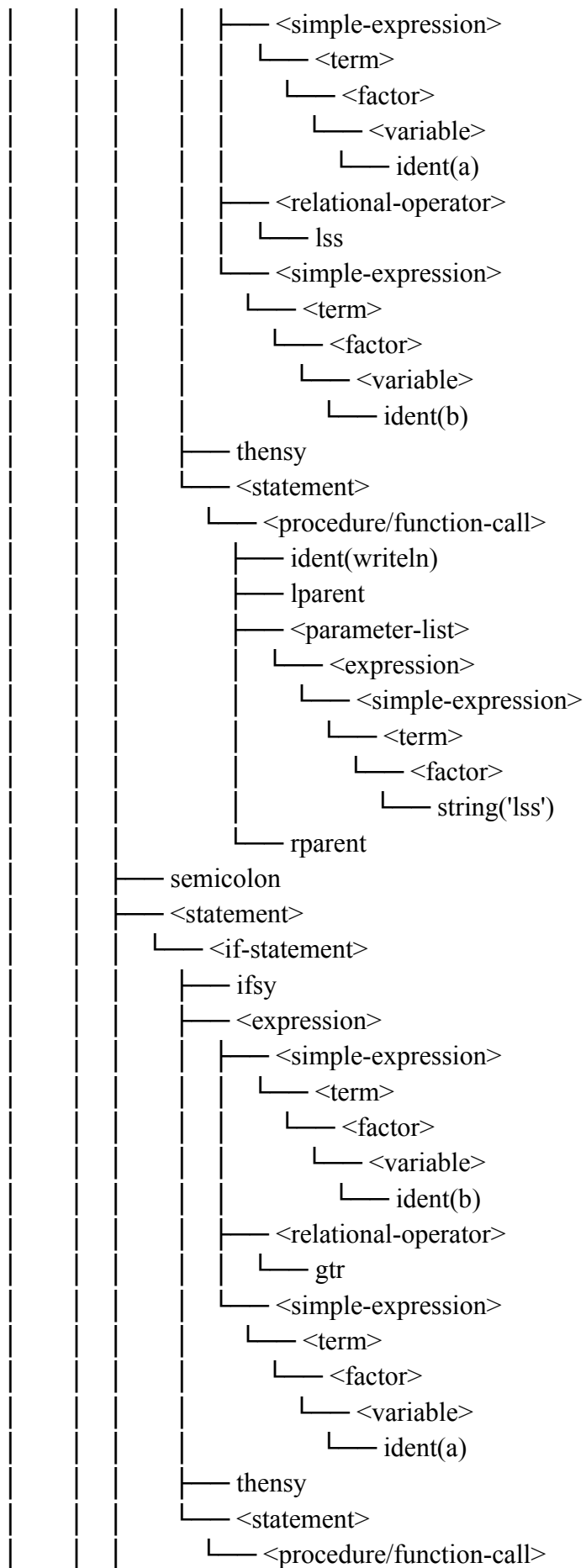
endsy
period

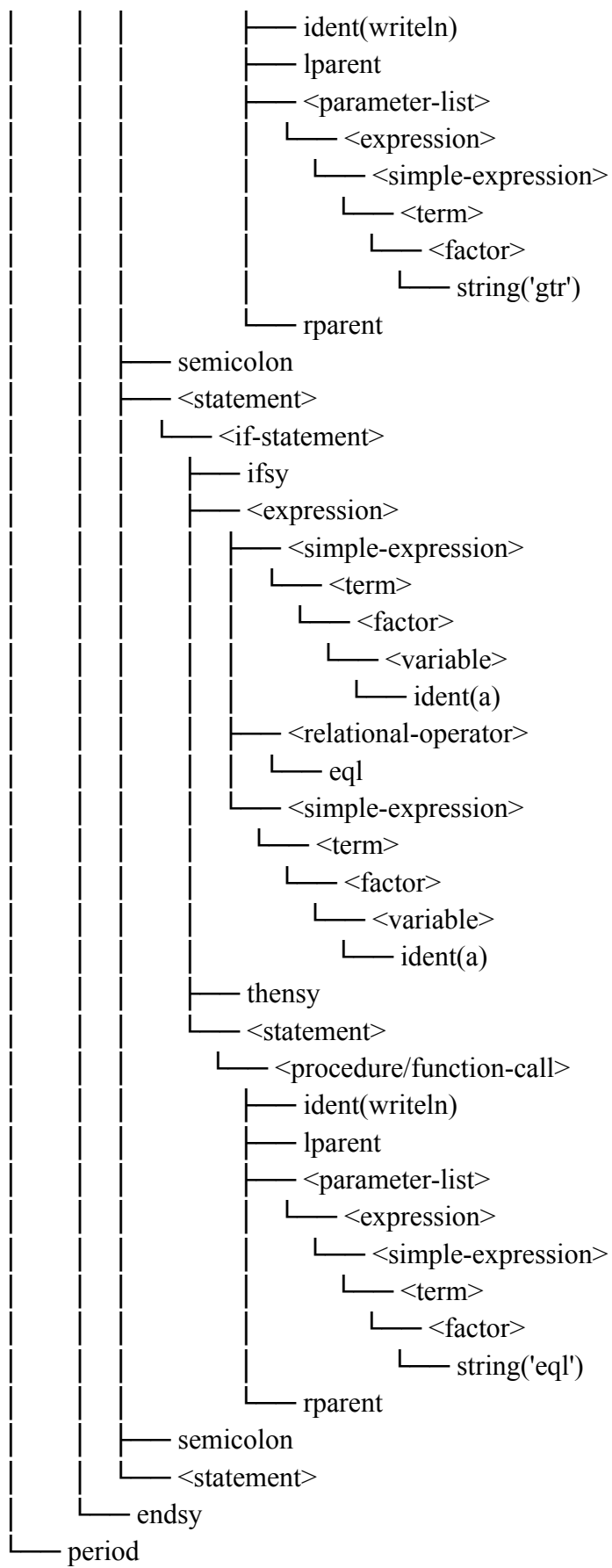
--- PARSE TREE ---
<program>











Bab 4. Kesimpulan dan Saran

Dengan memanfaatkan konsep Deterministic Finite Automata, kami telah berhasil merancang dan mengimplementasikan lexical analyzer yang dapat mengenali pola-pola token secara sistematis dan deterministik. Program yang telah kami buat mampu mengidentifikasi beberapa jenis token seperti *keyword*, *identifier*, *integer constant*, *real constant*, *string*, *character*, serta *operator* dan *delimiter*. Selebihnya, program mampu mengenali dan mengabaikan elemen yang tidak diperlukan seperti komentar *curly braces*, komentar *parentheses-star* dan spasi.

Dari kelima uji kasus, output menunjukkan bahwa:

- Program mampu mengenali perbedaan *spacing* dan *case sensitivity*.
- Program dapat mengenali berbagai bentuk literal seperti *string* dengan karakter *escape*
- Program dapat mengenali operator *relational* dan aritmetika secara tepat
- Program dapat mengabaikan komentar dalam beberapa format tanpa mempengaruhi hasil tokenisasi

Sehingga dapat disimpulkan bahwa program lexical analyzer telah diimplementasikan dengan baik dan sesuai dengan teori penggunaan DFAA sebagai model pemrosesan input

Dengan memanfaatkan konsep Recursive Descent Parsing dan Context-Free Grammar, kami juga telah berhasil merancang dan mengimplementasikan parser yang dapat menganalisis struktur sintaksis program berbahasa Arion secara hierarkis. Parser yang diimplementasikan mengikuti paradigma LL(k), di mana keputusan produksi dibuat berdasarkan token saat ini dan sejumlah token lookahead ke depan menggunakan fungsi `peekAt()`. Seluruh grammar di-hardcode sebagai sekumpulan fungsi rekursif, dengan masing-masing non-terminal direpresentasikan oleh satu fungsi tersendiri, sehingga struktur program mencerminkan struktur grammar secara langsung.

Dari kelima uji kasus, output menunjukkan bahwa:

- Program mampu membangun parse tree yang benar sesuai grammar bahasa Arion untuk berbagai konstruksi sintaksis, mulai dari deklarasi variabel, ekspresi aritmetika, hingga pernyataan kondisional.
- Program dapat membedakan assignment statement dan procedure/function call menggunakan mekanisme lookahead tanpa backtracking.
- Program menghasilkan pesan error sintaksis yang informatif, menyebutkan nomor baris, token yang ditemukan, dan token yang diharapkan.
- Parse tree ditampilkan dengan format visual yang jelas menggunakan karakter box-drawing, baik di terminal maupun dalam file output .txt.

Sehingga dapat disimpulkan bahwa program parser telah diimplementasikan dengan baik dan sesuai dengan teori penggunaan Recursive Descent Parsing sebagai metode analisis sintaksis top-down.

Setelah pertimbangan lebih, kami merumuskan bahwa terdapat ruang yang dapat dikembangkan lebih lanjut, yakni:

- **Pemulihan Error (Error Recovery)**

Sistem dapat dikembangkan agar dapat memberikan pesan kesalahan yang informatif seperti token yang tidak dikenal atau format literal yang tidak valid. Saat ini parser menghentikan proses begitu menemukan error pertama (fail-fast). Sistem dapat dikembangkan agar mampu melanjutkan parsing setelah menemukan error menggunakan teknik panic-mode recovery, sehingga lebih dari satu error sintaksis dapat dilaporkan sekaligus dalam satu kali proses.

- **Validasi Semantik Awal**

Saat ini parser hanya memeriksa kebenaran sintaksis. Tahap berikutnya dapat menambahkan pemeriksaan semantik sederhana seperti pengecekan apakah variabel yang digunakan sudah dideklarasikan sebelumnya.

- **Dukungan Grammar yang Lebih Lengkap**

Beberapa konstruksi bahasa Arion yang lebih kompleks seperti nested procedure/function call sebagai ekspresi dalam argumen dan tipe record bersarang dapat dikembangkan lebih lanjut untuk mendukung program yang lebih kompleks.

- **Optimasi Performa**

Masih terdapat ruang untuk mengoptimalkan implementasi DFA untuk menangani input berukuran besar.

Lampiran

Tautan Repository Github: <https://github.com/Beeziebloop/ZZZ-Tubes-IF2224-2026>

Pembagian Tugas:

NIM	Nama	Pembagian Tugas
13524125	Muhammad Rafi Akbar	parse_compound_statement()) parse_statement_list() parse_statement() parse_assignment_statement()) parse_if_statement() parse_while_statement() parse_repeat_statement() parse_for_statement() parse_case_statement() + parse_case_block() parse_procedure_function_call() + parse_parameter_list(), laporan
13524135	Varistha Devi	Inisialisasi Parser, parse_expression() parse_simple_expression() parse_term() parse_factor() parse_relational_operator() parse_additive_operator() parse_multiplicative_operator()) Parse Tree node struct/class (TokenNode, NonTerminalNode, dll.) printTree() / output ke file Token consumer (expect(), peek(), advance(), error handling). laporan
13524138	Ahmad Rinofaros Muchtar	parse_program() → root parse_program_header() parse_declaration_part() parse_const_declaration() + parse_constant() parse_type_declaration() + parse_type() + parse_array_type() +

		parse_range() + parse_enumerated() + parse_record_type() + parse_field_list() + parse_field_part() parse_var_declaration() + parse_identifier_list() parse_subprogram_declaration() → parse_procedure_declaration() + parse_function_declaration() parse_formal_parameter_list() + parse_parameter_group() parse_block(), Laporan
--	--	--

Referensi

1. <https://www.geeksforgeeks.org/compiler-design/introduction-of-parsing-ambiguity-and-parsers-set-1/>
2. <https://www.ibm.com/docs/en/integration-bus/10.0.0?topic=overview-parsers>
3. <https://fitech101.aalto.fi/en/courses/modern-and-emerging-programming-languages/part-2/4-a-step-back-lexers-and-parsers>
4. https://www.tutorialspoint.com/automata_theory/automata_theory_parse_tree.htm
5. <https://www.slideshare.net/slideshow/recursive-descent-parsing-66771172/66771172#4>
6. <https://www.geeksforgeeks.org/compiler-design/recursive-descent-parser/>
7. <https://www.geeksforgeeks.org/compiler-design/classification-of-top-down-parsers/>
8. <https://courses.cs.duke.edu/spring07/cps140/lects/sect11parseS.pdf>
9. https://www.youtube.com/watch?v=_VGVfkp9zjs